

Homework 03

Steven Jia

September 13, 2026

Question 1

I made a short skill called explain-to-me. I saved it in its own folder, checked its format, and installed the same file in my personal Codex skills folder.

For the test, I explicitly asked it to read Question 2. It explained the fixed-design simulation, listed what each part asks for, pointed me to the lecture section called “A controlled experiment: stability under correlation,” and suggested separating the quantities made once from the quantities made in every repetition. It ended by asking what was still unclear. It did not calculate anything or write solution code, so I kept the installed version. The complete skill file is at the very end of this homework.

Question 2

(a) Design check

I generated the design once and standardized with the average squared value, not the usual sample standard deviation.

```
set.seed(43231)

n <- 100
p <- 4
B <- 200

L <- rnorm(n)
U1 <- rnorm(n)
U2 <- rnorm(n)
X <- cbind(
  L + 0.04 * U1,
  L + 0.04 * U2,
  rnorm(n),
  rnorm(n)
)

X <- sweep(X, 2, colMeans(X), "-")
X <- sweep(X, 2, sqrt(colMeans(X^2)), "/")

singular_values <- svd(X, nu = 0, nv = 0)$d
design_check <- data.frame(
  correlation = cor(X[, 1], X[, 2]),
  largest_singular_value = max(singular_values),
  smallest_singular_value = min(singular_values)
)

knitr::kable(design_check, digits = 4)
```

correlation	largest_singular_value	smallest_singular_value
0.9977	14.1415	0.4754

The first two columns have correlation about 0.998. They move together, so the sum direction has a lot of information. Their difference is almost zero and is close to the weakest singular direction. That is why the data can estimate $\hat{\beta}_1 + \hat{\beta}_2$ much better than $\hat{\beta}_1 - \hat{\beta}_2$.

(b) Repeated responses

I kept this same X . Inside each repetition I made one training response and one independent test response, then reused that pair for all three penalties.

```

beta <- c(1.5, 1.5, 1, 0)
mu <- drop(X %*% beta)
lambda_grid_q2 <- c(0, 0.02, 0.2)

beta_hat <- array(
  NA_real_,
  dim = c(B, p, length(lambda_grid_q2))
)
train_mse_q2 <- test_mse_q2 <- matrix(
  NA_real_,
  nrow = B,
  ncol = length(lambda_grid_q2)
)

for (b in seq_len(B)) {
  y <- mu + rnorm(n)
  y_test <- mu + rnorm(n)
  y_bar <- mean(y)
  y_centered <- y - y_bar

  for (l in seq_along(lambda_grid_q2)) {
    lambda <- lambda_grid_q2[l]
    beta_hat[b, , l] <- solve(
      crossprod(X) + n * lambda * diag(p),
      crossprod(X, y_centered)
    )
    y_hat <- y_bar + drop(X %*% beta_hat[b, , l])
    train_mse_q2[b, l] <- mean((y - y_hat)^2)
    test_mse_q2[b, l] <- mean((y_test - y_hat)^2)
  }
}

coefficient_rows <- lapply(
  seq_along(lambda_grid_q2),
  function(l) {
    b1 <- beta_hat[, 1, l]
    b2 <- beta_hat[, 2, l]
    data.frame(
      lambda = lambda_grid_q2[l],
      beta1 = sprintf("%.3f (%.3f)", mean(b1), sd(b1)),
      beta2 = sprintf("%.3f (%.3f)", mean(b2), sd(b2)),
      sum = sprintf(
        "%.3f (%.3f)", mean(b1 + b2), sd(b1 + b2)
      ),
      difference = sprintf(
        "%.3f (%.3f)", mean(b1 - b2), sd(b1 - b2)
      )
    )
  }
)

```

```

    )
  }
)
coefficient_summary <- do.call(rbind, coefficient_rows)

error_summary_q2 <- data.frame(
  lambda = lambda_grid_q2,
  training_mse = colMeans(train_mse_q2),
  test_mse = colMeans(test_mse_q2)
)

knitr::kable(
  coefficient_summary,
  caption = "Mean (standard deviation) over 200 repetitions"
)

```

Table 1: Mean (standard deviation) over 200 repetitions

lambda	beta1	beta2	sum	difference
0.00	1.496 (1.544)	1.510 (1.549)	3.005 (0.095)	-0.014 (3.091)
0.02	1.487 (0.162)	1.489 (0.166)	2.976 (0.094)	-0.002 (0.314)
0.20	1.368 (0.046)	1.369 (0.047)	2.737 (0.086)	-0.001 (0.034)

```

knitr::kable(
  error_summary_q2,
  digits = 3,
  caption = "Average training and test MSE"
)

```

Table 2: Average training and test MSE

lambda	training_mse	test_mse
0.00	0.965	1.062
0.02	0.975	1.049
0.20	1.077	1.133

(c) Bias and variance

OLS gets the two individual coefficients right on average, but their standard deviations are both around 1.5. Their difference is even more unstable, with standard deviation about 3.1. At the same time, the sum has standard deviation only about 0.1.

With $\lambda = 0.02$, the individual standard deviations fall a lot and the estimated sum stays close to its true value of 3. This small amount of bias is worth the variance reduction here, and its average test MSE is a little lower than OLS. With $\lambda = 0.2$, the variance falls again, but the sum is pulled down too much. The extra bias makes test MSE worse.

The fitted values mostly depend on the well-observed sum direction. Ridge can remove a lot of movement in the weak difference direction without changing $X\hat{\beta}$ by the same amount.

Question 3

(a) Normal equation and uniqueness

Differentiating the objective gives

$$\nabla L_{\lambda}(\beta) = -\frac{1}{n}\mathbf{X}^{\top}(\tilde{\mathbf{y}} - \mathbf{X}\beta) + \lambda\beta.$$

Setting this equal to zero and multiplying by n gives

$$(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \hat{\beta}_\lambda = \mathbf{X}^\top \tilde{\mathbf{y}}.$$

For any nonzero vector $\mathbf{a}$,

$$\mathbf{a}^\top (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \mathbf{a} = \|\mathbf{X}\mathbf{a}\|_2^2 + n\lambda \|\mathbf{a}\|_2^2 > 0$$

when $\lambda > 0$. So the matrix is positive definite and invertible. This is still true when X is rank deficient or $p > n$, which makes the ridge solution unique.

(b) Singular directions

Using the compact SVD, the slope estimate can be written as

$$\hat{\beta}_\lambda = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}.$$

Multiplying by X gives the fitted centered response

$$\mathbf{X} \hat{\beta}_\lambda = \sum_{j=1}^r \frac{d_j^2}{d_j^2 + n\lambda} \mathbf{u}_j \mathbf{u}_j^\top \tilde{\mathbf{y}}.$$

So the factor in direction j is $\rho_j(\lambda) = d_j^2 / (d_j^2 + n\lambda)$. Here $n\lambda = 1$, so

$$\rho_1 = \frac{100}{101} = 0.9901, \quad \rho_2 = \frac{4}{5} = 0.8000, \quad \rho_3 = \frac{0.04}{1.04} = 0.0385.$$

The effective degrees of freedom, including the intercept, are

$$\text{df}_{\text{eff}} = 1 + 0.9901 + 0.8000 + 0.0385 = 2.8286.$$

If there are coefficient directions in the null space of X , ridge sets them to zero because they cannot change the fitted response but would increase the penalty.

(c) Strongest shrinkage

The third direction gets the strongest shrinkage because $d_3 = 0.2$ is the smallest singular value. The data have little information there, so noise can cause a large unpenalized estimate. Shrinking that direction can lower prediction variance. It can also make a lot of bias if the true mean response actually has a large component in that same direction.

Question 4

(a) Condition numbers

Without the penalty,

$$\kappa(\mathbf{A}_0) = \frac{9}{0.01} = 900.$$

Ridge changes the eigenvalues to 9.09 and 0.10, giving

$$\kappa(\mathbf{A}_\lambda) = \frac{9.09}{0.10} = 90.9.$$

The curvature is still different across the two directions, but much less different than before. The objective contours become less long and thin, so the optimization problem is better conditioned.

(b) Gradient descent

For a quadratic ridge objective,

$$\nabla L_\lambda(\beta^{(k)}) = \mathbf{A}_\lambda(\beta^{(k)} - \hat{\beta}_\lambda) = \mathbf{A}_\lambda \mathbf{e}^{(k)}.$$

It follows that

$$\mathbf{e}^{(k+1)} = (\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

An eigendirection with unpenalized eigenvalue a therefore gets multiplied by $1 - \eta(a + \lambda)$ at every step. Every direction contracts when

$$0 < \eta < \frac{2}{M} = \frac{2}{9.09} \approx 0.2200.$$

For $\eta = 1/M$, the strong-direction factor is $1 - 9.09/9.09 = 0$. Its error is gone in one step. The weak-direction factor is

$$1 - \frac{0.10}{9.09} \approx 0.9890,$$

so it moves much more slowly. For $\eta = 2.1/M$, the strong-direction factor is $1 - 2.1 = -1.1$. Its magnitude is bigger than one, so that error alternates sign and grows. With the nonzero starting component from the question, gradient descent does not converge.

(c) Statistical tuning is different

A larger λ can improve the condition number, but it also changes the estimator and adds bias. The step size and optimizer are chosen to solve a fixed objective. The penalty needs a statistical rule such as cross-validation or GCV, based on prediction rather than how fast the algorithm runs.

Question 5

(a) Ten-fold cross-validation

I matched the two files by row ID and used only the six predictors. In each validation split, the means and scales came from the other nine folds. The scale below uses a divisor of n , like the question asks.

```
realestate <- read.csv("data/realestate.csv")
split_folds <- read.csv("data/realestate-split-folds.csv")

stopifnot(
  anyDuplicated(realestate$row_id) == 0,
  anyDuplicated(split_folds$row_id) == 0,
  nrow(realestate) == nrow(split_folds)
)

match_index <- match(split_folds$row_id, realestate$row_id)
stopifnot(
  !anyNA(match_index),
  setequal(realestate$row_id, split_folds$row_id)
)

course_data <- cbind(
  split_folds,
  realestate[
    match_index,
    setdiff(names(realestate), "row_id"),
  ]
)
```

```

    drop = FALSE
  ]
)
stopifnot(identical(course_data$row_id, split_folds$row_id))

feature_names <- c(
  "date", "age", "distance",
  "stores", "latitude", "longitude"
)
X_raw_all <- as.matrix(course_data[, feature_names])
is_train <- course_data$split == "train"
X_train_raw <- X_raw_all[is_train, , drop = FALSE]
y_train <- course_data$price[is_train]
folds <- course_data$cv_fold[is_train]

stopifnot(
  nrow(X_train_raw) == 332,
  sum(!is_train) == 82,
  length(unique(folds)) == 10
)

lambda_grid <- c(0, 10^seq(-4, 2, length.out = 121))
fold_levels <- sort(unique(folds))
fold_mse <- matrix(
  NA_real_,
  nrow = length(fold_levels),
  ncol = length(lambda_grid)
)

for (k in seq_along(fold_levels)) {
  validation <- folds == fold_levels[k]
  X_fit_raw <- X_train_raw[!validation, , drop = FALSE]
  X_valid_raw <- X_train_raw[validation, , drop = FALSE]
  y_fit <- y_train[!validation]
  y_valid <- y_train[validation]

  x_bar <- colMeans(X_fit_raw)
  X_fit <- sweep(X_fit_raw, 2, x_bar, "-")
  x_scale <- sqrt(colMeans(X_fit^2))
  X_fit <- sweep(X_fit, 2, x_scale, "/")
  X_valid <- sweep(X_valid_raw, 2, x_bar, "-")
  X_valid <- sweep(X_valid, 2, x_scale, "/")

  y_bar <- mean(y_fit)
  y_centered <- y_fit - y_bar
  n_fit <- nrow(X_fit)
  p_fit <- ncol(X_fit)
  gram <- crossprod(X_fit)
  rhs <- crossprod(X_fit, y_centered)

  for (l in seq_along(lambda_grid)) {
    lambda <- lambda_grid[l]
    beta_hat <- solve(
      gram + n_fit * lambda * diag(p_fit),
      rhs
    )
    y_hat_valid <- y_bar + drop(X_valid %*% beta_hat)
    fold_mse[k, l] <- mean((y_valid - y_hat_valid)^2)
  }
}

```

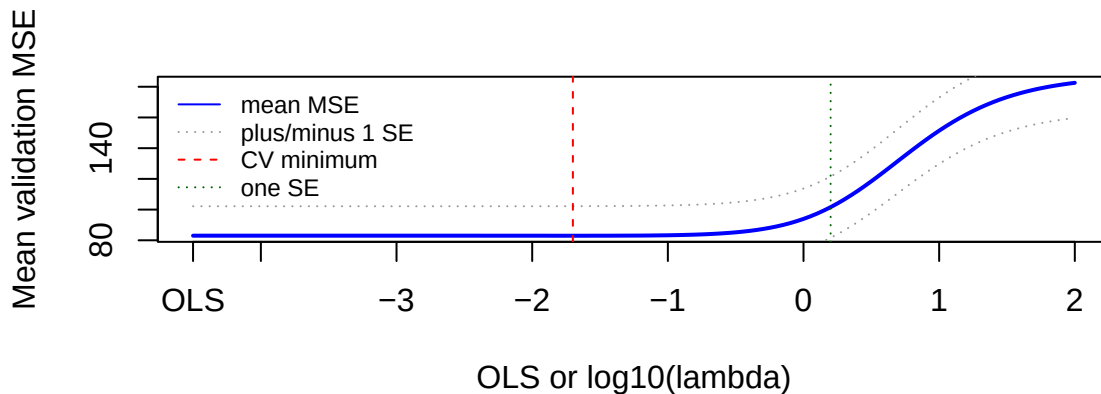
```

cv_mean <- colMeans(fold_mse)
cv_se <- apply(fold_mse, 2, sd) / sqrt(nrow(fold_mse))
min_index <- which.min(cv_mean)
lambda_min <- lambda_grid[min_index]
one_se_limit <- cv_mean[min_index] + cv_se[min_index]
lambda_1se <- max(lambda_grid[cv_mean <= one_se_limit])
one_se_index <- match(lambda_1se, lambda_grid)

plot_x <- c(-4.5, seq(-4, 2, length.out = 121))
penalty_position <- function(lambda) {
  if (lambda == 0) -4.5 else log10(lambda)
}

plot(
  plot_x, cv_mean,
  type = "l", col = "blue", lwd = 2,
  xaxt = "n",
  xlab = "OLS or log10(lambda)",
  ylab = "Mean validation MSE"
)
axis(1, at = c(-4.5, -4:2), labels = c("OLS", -4:2))
lines(plot_x, cv_mean + cv_se, col = "grey60", lty = 3)
lines(plot_x, cv_mean - cv_se, col = "grey60", lty = 3)
abline(
  v = penalty_position(lambda_min),
  col = "red", lty = 2
)
abline(
  v = penalty_position(lambda_1se),
  col = "darkgreen", lty = 3
)
legend(
  "topleft",
  c("mean MSE", "plus/minus 1 SE", "CV minimum", "one SE"),
  col = c("blue", "grey60", "red", "darkgreen"),
  lty = c(1, 3, 2, 3),
  bty = "n", cex = 0.72
)

```



```

cv_choices <- data.frame(
  rule = c("CV minimum", "One SE"),
  lambda = c(lambda_min, lambda_1se),

```

```

mean_validation_mse = c(
  cv_mean[min_index],
  cv_mean[one_se_index]
)
)
knitr::kable(cv_choices, digits = 5)

```

rule	lambda	mean_validation_mse
CV minimum	0.01995	82.97765
One SE	1.58489	101.59553

```
cat("One-SE limit:", round(one_se_limit, 5), "\n")
```

```
## One-SE limit: 102.2076
```

The minimum is at $\lambda = 0.01995$. The one-SE limit is fairly wide, so the largest allowed value is much bigger, $\lambda_{1se} = 1.58489$. Both choices were fixed without using the test responses.

(b) GCV

For GCV I standardized once using all 332 training observations. The effective degrees of freedom include one more for the intercept.

```

x_bar <- colMeans(X_train_raw)
X_train <- sweep(X_train_raw, 2, x_bar, "-")
x_scale <- sqrt(colMeans(X_train^2))
X_train <- sweep(X_train, 2, x_scale, "/")

y_bar <- mean(y_train)
y_centered <- y_train - y_bar
n_train <- nrow(X_train)
p_train <- ncol(X_train)
gram <- crossprod(X_train)
rhs <- crossprod(X_train, y_centered)
singular_values <- svd(X_train, nu = 0, nv = 0)$d

gcv <- df_eff_path <- numeric(length(lambda_grid))
for (l in seq_along(lambda_grid)) {
  lambda <- lambda_grid[l]
  beta_hat <- solve(
    gram + n_train * lambda * diag(p_train),
    rhs
  )
  y_hat <- y_bar + drop(X_train %*% beta_hat)
  df_eff_path[l] <- 1 + sum(
    singular_values^2 /
    (singular_values^2 + n_train * lambda)
  )
  training_mse <- mean((y_train - y_hat)^2)
  gcv[l] <- training_mse /
    (1 - df_eff_path[l] / n_train)^2
}

gcv_index <- which.min(gcv)
lambda_gcv <- lambda_grid[gcv_index]

plot(

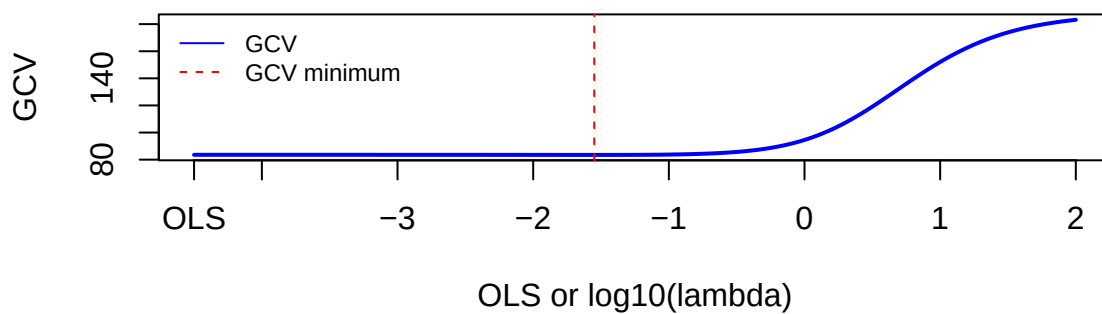
```



```

plot_x, gcv,
type = "l", col = "blue", lwd = 2,
xaxt = "n",
xlab = "OLS or log10(lambda)",
ylab = "GCV"
)
axis(1, at = c(-4.5, -4:2), labels = c("OLS", -4:2))
abline(
  v = penalty_position(lambda_gcv),
  col = "red", lty = 2
)
legend(
  "topleft",
  c("GCV", "GCV minimum"),
  col = c("blue", "red"),
  lty = c(1, 2),
  bty = "n", cex = 0.75
)

```



```

gcv_choice <- data.frame(
  rule = "GCV",
  lambda = lambda_gcv,
  GCV = gcv[gcv_index]
)
knitr::kable(gcv_choice, digits = 5)

```

rule	lambda	GCV
GCV	0.02818	83.50191

GCV picks $\lambda_{\text{GCV}} = 0.02818$, which is close to the CV minimum. Both rules use only training information and suggest fairly light shrinkage.

(c) Final comparison

I put the three selected values into the table first. Only after that did I take out the final test responses and calculate test MSE.

```

fit_ridge <- function(X_raw, y, lambda, X_raw_new) {
  n <- nrow(X_raw)
  p <- ncol(X_raw)

```

```

x_bar <- colMeans(X_raw)
X <- sweep(X_raw, 2, x_bar, "-")
x_scale <- sqrt(colMeans(X^2))
X <- sweep(X, 2, x_scale, "/")

y_bar <- mean(y)
y_centered <- y - y_bar
beta_hat <- solve(
  crossprod(X) + n * lambda * diag(p),
  crossprod(X, y_centered)
)

beta_raw <- drop(beta_hat) / x_scale
intercept_raw <- y_bar - sum(x_bar * beta_raw)
y_hat <- intercept_raw + drop(X_raw %*% beta_raw)
y_hat_new <- intercept_raw + drop(X_raw_new %*% beta_raw)

d <- svd(X, nu = 0, nv = 0)$d
df_eff <- 1 + sum(d^2 / (d^2 + n * lambda))

list(
  y_hat = y_hat,
  y_hat_new = y_hat_new,
  df_eff = df_eff
)
}

selected <- data.frame(
  rule = c("OLS", "CV minimum", "One SE", "GCV"),
  lambda = c(0, lambda_min, lambda_1se, lambda_gcv)
)

# The choices above are now frozen.
X_test_raw <- X_raw_all[!is_train, , drop = FALSE]
y_test <- course_data$price[!is_train]

selected$df_eff <- NA_real_
selected$train_mse <- NA_real_
selected$test_mse <- NA_real_

for (l in seq_len(nrow(selected))) {
  fit <- fit_ridge(
    X_train_raw,
    y_train,
    selected$lambda[l],
    X_test_raw
  )
  selected$df_eff[l] <- fit$df_eff
  selected$train_mse[l] <- mean(
    (y_train - fit$y_hat)^2
  )
  selected$test_mse[l] <- mean(
    (y_test - fit$y_hat_new)^2
  )
}

knitr::kable(
  selected, digits = c(0, 5, 3, 3, 3),
  caption = "Final training and test comparison"
)

```

Table 3: Final training and test comparison

rule	lambda	df_eff	train_mse	test_mse
OLS	0.00000	7.000	80.094	66.322
CV minimum	0.01995	6.773	80.135	66.000
One SE	1.58489	3.033	100.374	81.200
GCV	0.02818	6.689	80.171	65.903

OLS has the smallest training MSE, which is expected because it does not shrink the slopes. The CV-minimum and GCV fits use mild shrinkage and have almost the same errors and effective degrees of freedom. The one-SE fit is much smaller in effective degrees of freedom and has more bias, so its MSE is larger on this test set.

GCV happens to have the smallest test MSE in the table, but I cannot switch to it because of that result. The three rules were already fixed. Choosing again after looking at the test numbers would turn the test set into validation data and make the reported minimum too optimistic. GCV itself is still a training-based estimate, not another independent test set.

explain-to-me skill

```

---
name: explain-to-me
description: Use only when I explicitly ask for help understanding a STAT 432 homework question without solving it.
---

# Intent

Help me unpack what a homework question wants while leaving the actual solution work for me.

# Instructions

- Read the whole question before explaining it.
- In short, conversational language, state the goal, the required output, the main notation, and any important constraints.
- Keep the notation from the question instead of replacing it with new symbols.
- For Questions 2 and 3, consult [Ridge Regression: Stability Through Shrinkage](https://teazrq.github.io/stat432rpy/topics/ridge-regression/ridge-regression.html). For Questions 4 and 5, consult [From a Penalized Objective to a Fitted Ridge Model](https://teazrq.github.io/stat432rpy/topics/ridge-regression/optimization-and-cross-validation.html). Name one useful section and briefly say why it helps.
- Suggest one sensible first step, then ask which part still feels unclear.
- Do not do calculations or derivations, write solution code, or provide a final answer.
```